

Hybrid IaC Misconfiguration Detection

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Vishvaksen
Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Hybrid IaC Misconfiguration Detection

Vishvaksen

TBD

Abstract

War et al. (2025), “Detection of Security Smells in IaC Scripts through Semantics-Aware Code and Language Processing” (University of Luxembourg, arXiv:2509.18790; a preprint whose peer-review status could not be confirmed), fine-tune CodeBERT/LongFormer to detect misconfigurations in Ansible/Puppet scripts, substantially beating prior TF-IDF-based baselines. Their own ablation study is the source of the gap this project targets: removing natural-language context (comments, task descriptions) causes precision to collapse sharply while recall holds or rises – CodeBERT on Ansible drops from precision 0.92 to 0.46 (F1 0.90 to 0.58). This project reproduces that qualitative collapse on a synthetic IaC benchmark deliberately split into “easy” misconfigurations (safe/unsafe distinguishable from code tokens alone) and “hard” ones (the code is lexically identical either way; only an explanatory comment reveals the label) – a lightweight TF-IDF + logistic regression classifier drops from precision/recall/F1 of 1.00/1.00/1.00 with comments to 0.77/0.71/0.73 without them. A hybrid detector is then built, combining a comment-independent rule layer for the enumerable patterns with the same classifier gated at a stricter confidence threshold. It restores perfect precision (1.00) in the code-only setting – matching the rich-context ideal – but at a real, disclosed recall cost (0.48 vs. the plain classifier’s 0.71): the “hard” pattern is by construction unsolvable without the missing comment, and no rule/ML combination recovers information that was never present in the code. Sweeping the confidence threshold (0.5 to 0.9) shows this is a tunable precision/recall trade-off, not a single fixed outcome.

1 Introduction

Infrastructure-as-Code (IaC) misconfigurations – hardcoded secrets, overly permissive file modes, unencrypted protocols – are a recurring cause of real security incidents. Rule-based scanners like Checkov catch well-known, enumerable patterns reliably, and recent work has shown that semantics-aware ML models (fine-tuned on both code and accompanying natural-language comments) can substantially outperform older statistical baselines at this task.

War et al. War et al. (2025) demonstrate exactly this with CodeBERT and LongFormer, but their own ablation study exposes a specific weakness: when natural-language context is stripped away, precision collapses while recall holds or rises, meaning the model starts flagging large amounts of safe code as misconfigured. This matters in practice, since real-world IaC repositories are often sparsely commented, unlike the curated, comment-rich datasets used to train and evaluate their models.

Research question: can a hybrid detector – combining a comment-independent rule layer with the same ML classifier, gated more conservatively – recover the precision lost when natural-language context is unavailable, and if so, at what cost?

Objectives:

- Reproduce the paper’s ablation finding (precision collapse without natural-language context) on a controlled synthetic benchmark.
- Design the benchmark so that some misconfigurations are code-alone-solvable (“easy”) and some are only resolvable via a natural-language comment (“hard”), making the reproduction genuine rather than trivially avoidable.
- Build a hybrid rule+ML detector targeting the gap, and report its actual trade-offs rather than only the metrics that look favourable.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the gap it names; Section 4 describes the easy/hard benchmark design and the three detectors compared; Section 5 describes the codebase and deployment; Section 6 reports results across 5 seeds and a threshold sweep; Section 7 discusses the trade-off and future work.

2 Related Work

Static analysis tools for Infrastructure-as-Code (IaC) such as Checkov and tfsec catch a well-known set of syntactically enumerable misconfigurations (permissive file modes, unencrypted protocols, world-open network rules) reliably, but struggle with misconfigurations that depend on context or intent rather than a fixed pattern.

2.1 Baseline: Semantics-Aware Detection with Code and Language Models

The baseline for this project is War et al. War et al. (2025), “Detection of Security Smells in IaC Scripts through Semantics-Aware Code and Language Processing” (University of Luxembourg, arXiv:2509.18790, September 2025). *This is cited as a preprint; its peer-review status could not be confirmed at the time of writing, unlike the baselines used elsewhere in this thesis series.* The paper fine-tunes CodeBERT (for Ansible scripts, which are rich in natural-language comments and task descriptions) and LongFormer (for long Puppet scripts) to classify IaC snippets as misconfigured or safe, substantially outperforming prior TF-IDF/Bag-of-Words + Random Forest baselines (e.g. Ansible F1 0.90 vs. 0.77) and performing competitively with much larger general-purpose LLMs.

2.2 The Gap This Project Targets

The paper’s own ablation study (their RQ2) is the source of the gap this project targets. Removing natural-language context – stripping comments from Ansible snippets, or truncating surrounding code for Puppet snippets – causes precision to collapse sharply while recall holds steady or rises: CodeBERT on Ansible drops from precision 0.92 to 0.46 (F1 0.90 to 0.58); LongFormer on Puppet drops from 0.87 to 0.55 (F1 0.79 to 0.70). Their own words: “without sufficient semantic context... models over-flag and lose precision,

undermining practical usability.” This is a real deployment concern: much real-world IaC is sparsely commented, unlike the curated, comment-rich datasets used to train and validate the paper’s models.

2.3 Hybrid Rule-Based and ML Detection

Combining rule-based static analysis with ML classifiers is a natural response to this specific failure mode – a fixed rule cannot lose precision due to missing comments, because it never used comments to begin with. This project builds and evaluates exactly that combination on a synthetic benchmark designed so that some misconfigurations are enumerable (rule-detectable) and some are only resolvable via natural-language context (structurally rule-proof), to test how much of the gap a hybrid approach can close, and to make explicit what it structurally cannot.

3 Methodology

3.1 Synthetic Benchmark Generation

For each of 5 seeds (42-46), 1,200 IaC snippets are generated (`src/data/iac_dataset.py`), each independently choosing an easy or hard pattern (50/50) and a safe/misconfigured label (50/50), with 2-4 unrelated filler configuration lines added as noise. Two paired renderings are produced from the identical underlying random choices – with comments and without – so that any difference in a model’s performance between the two is attributable only to the presence or absence of the comment, not to a different underlying sample.

3.2 Training and Evaluation

A 70/30 stratified train/test split is used per seed. The ML detector is trained separately on the rich-context and code-only training sets, then evaluated on the matching test set. The rule-based and hybrid detectors require no training beyond the ML component the hybrid detector wraps (trained on the code-only training set, since that is the setting the improvement targets).

3.3 Metrics

Precision, Recall, and F1 – the same three metrics War et al. use – computed on the held-out test set for each of: ML Detector (rich context), ML Detector (code-only), Rule-Based Only, and Hybrid Detector (code-only). A confidence-threshold sweep (0.5 to 0.9) for the hybrid detector’s ML fallback is also reported, to characterise the precision/recall trade-off rather than a single fixed operating point.

All code and results are in `iac-security/`.

4 Design Specification

4.1 Easy and Hard Misconfiguration Patterns

Each synthetic IaC snippet uses one of two pattern families:

- **Easy** (4 patterns): safe and misconfigured versions use visibly different, enumerable literal values – file mode `0777` vs. `0640`, protocol `http` vs. `https`, hash algorithm `sha1` vs. `sha256`, ingress CIDR `0.0.0.0/0` vs. a restricted range. Code tokens alone are genuinely sufficient to classify these.
- **Hard** (2 patterns): an opaque reference token (`password_ref = "ref_12345"`) whose numeric suffix carries no signal and is identical in shape whether safe or misconfigured. Only an accompanying comment (e.g. “resolves to a hardcoded fallback password” vs. “resolves via the managed secrets vault”) reveals the true label. No regex or code-only classifier can distinguish these by construction.

Each generated sample is rendered in two paired forms from the same underlying random choices: *with-comments* (rich context) and *code-only* (comments stripped) – mirroring War et al.’s own ablation axis.

4.2 ML Detector (baseline reproduction)

A TF-IDF vectorizer over snippet tokens feeding a logistic regression classifier – the same statistical-feature-based family as the paper’s own prior-work baseline (TF-IDF/BoW + Random Forest), used here in place of fine-tuned CodeBERT/LongFormer given no GPU/model-download budget for this project (see README scope note). Trained and evaluated separately on rich-context and code-only variants.

4.3 Rule-Based Detector (reference)

Four regular expressions matching the literal misconfigured value for each *easy* pattern, independent of any comment. Structurally cannot recognise the *hard* pattern at all – there is no fixed literal to match.

4.4 Hybrid Detector (proposed improvement)

Flags a snippet as misconfigured if the rule layer fires, *or* if the ML classifier’s predicted probability exceeds a stricter threshold (0.8, vs. a plain 0.5 cutoff). This is deliberately not a simple OR of “rule fires OR ML predicts positive at 0.5” – that would only ever add false positives on top of an already-low-precision code-only classifier. Gating the ML fallback at a stricter confidence threshold is what lets precision recover: unreliable, low-confidence guesses get filtered out rather than trusted.

5 Implementation

The system is implemented in Python 3 with scikit-learn. Key modules:

- `src/data/iac_dataset.py` – paired easy/hard, with/without-comment synthetic IaC snippet generator described in Section 3.
- `src/models/detectors.py` – `MLDetector` (baseline), `RULE_PATTERNS / rule_based_flag` (comment-independent rule layer), `HybridDetector` (improvement).

- `scripts/train_and_evaluate.py` – trains and evaluates all variants across 5 seeds, writes `results/results_per_seed.csv` and `results/results_summary.csv`, and exports the trained ML detector to `src/lambda_handler/model/` for deployment.
- `src/lambda_handler/app.py` – an AWS Lambda handler applying the hybrid detector to IaC snippets submitted on a Kinesis stream, returning whether a snippet looks misconfigured, whether a rule fired, and the ML classifier’s confidence.
- `template.yaml` – AWS SAM template provisioning the Kinesis stream and Lambda function.
- `.github/workflows/deploy.yml` (repo root) – CI pipeline that runs the training/evaluation script and, on push to `main`, deploys the SAM stack to the configured AWS region using repository secrets.

Running `python scripts/train_and_evaluate.py` reproduces every number in Section 6 from scratch. The Lambda handler was verified locally against representative safe and misconfigured snippets before deployment. An earlier, contaminated `Makefile` referencing an unrelated SQS project’s build targets was removed as part of this work.

6 Evaluation

Table 1 reports mean $\pm$ standard deviation across 5 seeds (42-46).

Table 1: Results across 5 seeds (mean $\pm$ std)

Model	Precision	Recall	F1
ML Detector (rich context)	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
ML Detector (code-only, baseline gap)	0.768 $\pm$ 0.045	0.708 $\pm$ 0.064	0.733 $\pm$ 0.019
Rule-Based Only (reference)	1.000 $\pm$ 0.000	0.483 $\pm$ 0.028	0.651 $\pm$ 0.025
Hybrid Detector (improved, code-only)	1.000 $\pm$ 0.000	0.483 $\pm$ 0.028	0.651 $\pm$ 0.025

6.1 Experiment 1: Reproducing the Precision Collapse

With comments present, the ML detector is perfect (1.00/1.00/1.00) – comment language directly names the risk (“WARNING: ...”), which is expected given how the benchmark is constructed. Without comments, precision falls to 0.768 and recall to 0.708, reproducing the qualitative direction of War et al.’s finding: removing natural-language context degrades a text classifier’s reliability, not just its confidence.

6.2 Experiment 2: The Hybrid Fix – and Its Real Cost

The hybrid detector restores precision to a perfect 1.000 in the code-only setting, matching the rich-context ideal. But its recall (0.483) is identical to the rule-based-only reference and *lower* than the plain code-only ML classifier’s 0.708. The reason is structural, not a tuning failure: the ML classifier contributes nothing beyond the rule layer at this threshold, because on “hard” pattern samples (which by construction carry no code-level signal) its predicted probability never exceeds 0.8 – there is nothing in the code for it to be confident about.

6.3 Experiment 3: The Trade-off Is Tunable, Not Fixed

Sweeping the hybrid detector’s confidence threshold traces a clear Pareto frontier:

Threshold	Precision	Recall	F1
0.5 (= plain ML)	0.768	0.708	0.733
0.6	0.928	0.528	0.672
0.7	0.998	0.485	0.652
0.8	1.000	0.483	0.651
0.9	1.000	0.483	0.651

Precision rises monotonically with the threshold and plateaus at 1.000 by 0.8; recall falls from the plain classifier’s 0.708 down to the rule-only floor of 0.483 and does not go lower, since the rule layer’s contribution is threshold-independent. An operator who cannot tolerate false positives should choose a high threshold; one who needs to catch more of the genuinely ambiguous cases, at the cost of some false positives, should choose a lower one. There is no threshold that achieves both – the hard pattern’s missing information cannot be substituted by a smarter combination rule.

6.4 Discussion

This project reproduces War et al.’s finding, builds a working fix for the specific case that fix can address (enumerable, rule-representable misconfigurations), and is explicit about what it cannot fix: misconfigurations whose safety genuinely depends on information that exists only in a comment cannot be recovered by any classifier or rule combination once that comment is gone. This mirrors the paper’s own conclusion that semantic context is not merely helpful but load-bearing – the honest implication is that comment/documentation quality in IaC repositories is itself a security control, not just a readability nicety.

7 Conclusion and Future Work

This project asked whether a hybrid rule+ML detector could recover the precision War et al. (2025) lose when natural-language context is stripped from IaC scripts, and at what cost. On a synthetic benchmark deliberately split into code-alone-solvable and comment-dependent misconfigurations, the hybrid detector fully restores precision (1.000, matching the rich-context ideal) in the code-only setting, but recall falls from the plain classifier’s 0.708 to 0.483 – exactly the rule layer’s ceiling, since the comment-dependent cases carry no code-level signal for any classifier or rule to find. A confidence-threshold sweep shows this is a genuine, tunable trade-off rather than a single verdict: more precision can always be bought with less recall on the ambiguous cases, but not both at once.

The honest conclusion is that War et al.’s own framing is right: semantic/natural-language context is not a nice-to-have for these models, it is load-bearing. A hybrid detector can make a system’s behaviour *safer* in the absence of that context (never worse than pure rules, and never as unreliable as an unfiltered low-context classifier), but it cannot substitute for the missing information.

Future work:

- Reproduce this on the paper’s actual Ansible/Puppet datasets with fine-tuned CodeBERT/LongFormer, to check whether the precision/recall trade-off curve has the same shape on real, non-synthetic misconfigurations.
- Extend the rule layer with a wider, community-maintained rule set (e.g. Checkov’s own policies) rather than four hand-written regexes, to test how much of the ”hard” category in real repositories is actually enumerable once a mature rule engine is used instead of a minimal one.
- Explore whether a model could be trained to *flag its own low-confidence, comment-dependent uncertainty* for human review, rather than only choosing between ”flag” and ”don’t flag” – turning the unresolvable cases into a routed queue instead of a silent miss.

References

War, A., Rawass, A. A., Kabore, A. K., Samhi, J., Klein, J. and Bissyandé, T. F. (2025). Detection of security smells in IaC scripts through semantics-aware code and language processing. Preprint, University of Luxembourg; peer-review status not confirmed at time of writing. <https://arxiv.org/abs/2509.18790>.